الملف السادس: واجهة لوحة التحكم (Dashboard) — معمارية الواجهة الأمامية بالكامل

أولاً: نظرة عامة على معمارية لوحة التحكم

لوحة التحكم مبنية كـ Single Page Application (SPA) باستخدام **React 18** و **TypeScript** و **Vite** للتجميع السريع. التصميم مبني على **Tailwind CSS** لضمان واجهة عصرية وسريعة الاستجابة.

تعتمد الواجهة في إدارة حالة البيانات (State Management) والاتصال بالخادم على مكتبة **TanStack Query (React Query)**. هذه المكتبة تُعدّ حجر الأساس في التطبيق لأنها تُعالج:

- **التخزين المؤقت (Caching):** تجنب جلب نفس البيانات مراراً.
- **التحديث التلقائي في الخلفية (Background Polling):** العديد من الصفحات تستخدم `refetchInterval` (مثل 8 ثوانٍ للأوامر، 30 ثانية للأجهزة) لتحديث البيانات حياً دون تدخل المستخدم.
- **معالجة الأخطاء وإعادة المحاولة:** إعادة محاولة الطلبات الفاشلة تلقائياً.

تتواصل الواجهة مع الخوادم (Connection Manager و Sigma Engine) عبر ملف `src/api/client.ts` الذي يستخدم `axios` مع Interceptors تُضيف `JWT Token` لكل طلب، وتُعالج أخطاء 401 بتسجيل خروج المستخدم التلقائي. كل الطلبات تذهب إلى Nginx (نفس الـ origin) الذي يُوجّهها للخدمة المناسبة خلف الكواليس لتجنب أخطاء CORS.

ثانياً: شرح مفصّل لجميع صفحات لوحة التحكم (Pages)

المشروع يحتوي على أكثر من 20 صفحة ومكوّن رئيسي في مجلد `/src/pages` . إليك شرح مفصل لكل صفحة، ماذا تفعل، ومن أين تأتي ببياناتها:

1. صفحة تسجيل الدخول (`Login.tsx`)

- **الوظيفة:** نقطة الدخول الوحيدة للنظام. تقبل اسم المستخدم وكلمة المرور.
- **تدفق البيانات:** تُرسل طلب `POST` إلى `api/v1/auth/login/` . في حال النجاح، تستلم `token` و `user` وتحفظهما في `localStorage` ، ثم تُوجّه المستخدم إلى الـ Dashboard.

2. الصفحة الرئيسية (`Dashboard.tsx`)

- **الوظيفة:** شاشة مراقبة العمليات (NOC/SOC View). تُعطي المحلل لمحة شاملة في ثوانٍ.
- **مكونات البيانات:**
  - بطاقات الإحصائيات (عدد الوكلاء، التنبيهات المفتوحة).
  - توزيع التنبيهات حسب الخطورة (Critical, High, Medium, Low).
  - مخطط زمني (Timeline) لآخر 24 ساعة يوضح ذروات الهجمات.
  - قائمة بأكثر الأجهزة خطورة (Top Vulnerable/Risky Endpoints).
- **تدفق البيانات:** تستخدم `agentsApi.stats()` لإحصائيات الأجهزة، و `alertsApi.list()` ، و `statsApi.timeline()` من محرك Sigma.

3. صفحة الأجهزة (`Endpoints.tsx`)

- **الوظيفة:** إدارة أسطول الأجهزة المحمية. تعرض جدولاً بجميع الوكلاء مع إمكانية البحث والفلترة حسب نظام التشغيل أو الحالة.
- **مكونات البيانات:** الاسم، نظام التشغيل، الحالة (Online/Offline/Isolated)، مقاييس الأداء (CPU/RAM)، وإصدار الوكيل.
- **تدفق البيانات:** تستدعي `agentsApi.list()` مع `refetchInterval` كل 30 ثانية لضمان دقة حالة الاتصال (Online/Offline).

4. صفحة تفاصيل الجهاز الشاملة (`EndpointDetail.tsx`)

- **الوظيفة:** أكبر صفحة في التطبيق (تحتوي على أكثر من 3000 سطر برمجي). هي مركز عمليات الجهاز الواحد.
- **التبويبات (Tabs) داخل الصفحة:**
  - **Overview (نظرة عامة):** ملخص لصحة الجهاز ومعلومات الشبكة والـ Sysmon. (تستخدم `agentsApi.get()`).
  - **Incident (الحادث):** تبويب يُظهر التنبيهات الخاصة بهذا الجهاز فقط والوضع الأمني الحالي له (يستدعي `alertsApi.list` مع `agent_id`).
  - **Response (الاستجابة والأوامر):** واجهة إصدار الأوامر للوكيل (عزل الشبكة، إيقاف عملية، فحص الذاكرة). تعرض تاريخ الأوامر وحالتها الحية (Pending -> Completed). (تستدعي `agentsApi.executeCommand()` و `agentsApi.getCommands()`).
  - **Quarantine (الحجر):** تعرض الملفات التي حجرها الوكيل محلياً. تتيح للمحلل استعادة الملف (Restore) أو حذفه نهائياً (Delete). (تستدعي `agentsApi.listQuarantine()`).
  - **Activity (النشاط):** بث مباشر لأحداث الجهاز (Events).
  - **Forensics (التحقيق الجنائي):** لاستعراض السجلات (Logs) التي جُمِعت بأمر `collect_logs` .
  - **Configuration:** لتعديل إعدادات الوكيل عن بُعد.
- **التحديث المباشر:** الصفحة تستخدم Polling مكثف (كل 5 إلى 8 ثوانٍ) لجداول الأوامر والحجر حتى تظهر نتيجة الأمر فور تنفيذه على الوكيل.

5. صفحة مركز الإجراءات (ActionCenter.tsx)

- **الوظيفة:** مراقبة الأوامر الصادرة لجميع الأجهزة في مكان واحد.
- **تدفق البيانات:** تجلب جميع الأوامر (agentsApi.getCommands()) مع إمكانية الفلترة (أوامر ناجحة، فاشلة، قيد التنفيذ). تُستخدم لمراقبة الاستجابات الجماعية للتهديدات.

6. صفحة التنبيهات (Alerts.tsx)

- **الوظيفة:** صندوق الوارد للمحلل الأمني. تعرض كل التنبيهات المُولّدة من Sigma Engine.
- **تدفق البيانات:** تستدعي alertsApi.list() . عند النقر على تنبيه، تُفتح تفاصيله الجانبية (Drawer) التي تعرض Confidence Score ، و Risk Score ، وحقول الأحداث المتطابقة، واقتراحات Playbooks للاستجابة السريعة.

7. صفحة إدارة قواعد سيجما (Rules.tsx)

- **الوظيفة:** واجهة إدارة مكتبة الكشف. تعرض القواعد النشطة، وتسمح بتفعيل/تعطيل قواعد معينة، أو إنشاء قواعد مخصصة جديدة.
- **تدفق البيانات:** تتعامل مع rulesApi.create , rulesApi.list() , و rulesApi.toggle() . البيانات تذهب مباشرة لـ Sigma Engine ويتم تحديثها في الـ RuleIndexer فوراً دون إعادة تشغيل المحرك.

8. صفحة إدارة الثغرات (VulnerabilityManagement.tsx)

- **الوظيفة:** عرض الثغرات المكتشفة (CVEs) في الأجهزة ومقاطعتها مع بيانات KEV (Known Exploited Vulnerabilities) من وكالة CISA لتحديد الأولويات.
- **تدفق البيانات:** تجلب vulnerabilityApi.listFindings() . الصفحة تقوم بعمل حسابات معقدة لإنتاج KPIs وعرض رسوم بيانية (باستخدام Recharts) لتوزيع خطورة الثغرات.

9. صفحة نشر الوكلاء (AgentDeployment.tsx و EnrollmentTokens.tsx)

- **الوظيفة:** إدارة دورة حياة النشر. توليد رموز التسجيل (Enrollment Tokens) وتحديد صلاحيتها وتوفير سكريبتات التثبيت الجاهزة.
- **تدفق البيانات:** تتواصل مع خدمة AgentService في Connection Manager لإصدار الرموز المؤقتة.

10. صفحات الأتمتة (automation/PlaybooksPage.tsx و automation/AutomationRulesPage.tsx)

- **Playbooks:** تصميم وتعديل "كتب اللعب" (ردود الفعل المبرمجة). مثلاً إنشاء Playbook باسم "Ransomware Containment" يحتوي على سلسلة من الأوامر.
- **Automation Rules:** ربط التنبيهات بـ Playbooks. (مثلاً: إذا كان التنبيه Critical ويحتوي على كلمة Ransomware، نفّذ Playbook العزل تلقائياً).
- **تدفق البيانات:** واجهات CRUD (إنشاء، قراءة، تحديث، حذف) تتصل بـ automationApi .

11. صفحة ارتباط الأحداث (`SocCorrelation.tsx`)

- **الوظيفة:** عرض الأحداث التي تم ربطها معاً زمنيّاً لتمثل سيناريو هجوم متقدم (مثلاً، تسجيل دخول فاشل ثم تحميل ملف مشبوه ثم تنفيذ عبر PowerShell).
- **تدفق البيانات:** تجلب بيانات الـ Correlation Cache المعتمدة على Redis.

12. صفحة سجلات التدقيق (`AuditLogs.tsx`)

- **الوظيفة:** الشفافية والمساءلة. عرض من فعل ماذا ومتى. من قفل الجهاز X؟ من ألغى قاعدة Y؟
- **تدفق البيانات:** تقرأ من جدول `audit_logs` في PostgreSQL لتلبية متطلبات الامتثال الأمني.

13. صفحة التهديدات والإحصائيات (`Threats.tsx` و `Stats.tsx`)

- **الوظيفة:** رسوم بيانية تحليلية لمدراء الـ SOC حول كفاءة النظام، معدلات الإسقاط، زمن الاستجابة، ونشاط التهديدات.

ثالثاً: الأسئلة الشاملة (Q&A) للجنة المناقشة

لقد جهزنا هذه الأسئلة بناءً على تعقيد المشروع المتكامل:

1. كيف تُحلّ مشكلة "انقسام الدماغ" (Split-Brain) إذا تغير عنوان الخادم أو فُصل الوكيل؟

الإجابة: الوكيل لا يعتمد على IP ثابت. بل يستخدم نظام DNS لحل عنوان C2. ثانياً، الوكيل مبرمج بـ Exponential Backoff. إذا فشل الاتصال، يحاول بعد ثانية، ثم ثانيتين، ثم 4، حتى يصل لـ 30 ثانية أقصى حد، ويستمر بالمحاولة للأبد بصمت. أثناء ذلك، يُخزّن كل الأحداث الأمنية في `Disk Queue WAL` (Write-Ahead Log). بمجرد عودة الاتصال، يعمل `Queue Processor` على إرسال كل الأحداث المتراكمة لضمان صفر فقدان للبيانات.

2. ماذا لو أرسل الوكيل 10,000 حدث في الثانية؟ ألن يتعطل الخادم؟

الإجابة: النظام مصمم للتعامل مع هذا الانفجار (Backpressure).

- على جانب الوكيل: توجد Channels بحجم محدود. إذا امتلأت، يتم عمل Rate Limiting وإسقاط الأحداث غير الحرجة للحفاظ على استقرار الـ CPU للعميل.
- على جانب الخادم: الـ gRPC يستقبلها ويضعها فوراً في Apache Kafka. Kafka مصمم لاستيعاب ملايين الرسائل في الثانية وتخزينها على القرص. محرك Sigma يقرأ من Kafka بالسرعة التي تناسبه. هذا الفصل (Decoupling) يمنع انهيار الخادم تحت الحمل العالي.

3. أرى ميزة "الاستجابة التلقائية" (Auto-Response)، كيف تضمنون ألا تعزل النظام أجهزة الإنتاج الحساسة (False Positives) بالخطأ؟

الإجابة: لقد بنينا طبقات متعددة من الأمان لمنع ذلك:

1. **الـ Risk Score:** التنبيه لن يُشغّل استجابة تلقائية إلا إذا تخطى الـ Risk Score الحد المسموح. درجة المخاطرة تنخفض تلقائياً إذا كان الجهاز يحمل علامة "Production" أو "Critical".
2. **الـ Cooldown Period:** إذا عملت القاعدة مرة، تُقفل (Cooldown) لعدة دقائق لمنع تكرار التنفيذ الخاطئ المتسلسل.
3. **ML Optimizer / Success Rate:** إذا فشل Playbook أو أظهر نسبة نجاح أقل من 50% تاريخياً، يتم إيقافه تلقائياً ويُطلب تدخل بشري.
4. **Whitelisting:** توجد قوائم استثناء للعمليات النظامية لا يُمكن إيقافها أبداً تحت أي ظرف.

4. كيف يمكن للمتصفح (Dashboard) أن يستلم تحديثاً بتنبيه جديد في أقل من ثانية دون تحديث الصفحة؟

الإجابة: نستخدم تقنية **WebSockets**. عندما يقوم Sigma Engine باكتشاف تنبيه ويضعه في Kafka، يلتقطه Connection Manager، ويقوم بنشره عبر Redis Pub/Sub إلى جميع نسخ الخادم. الخادم بدوره يُرسل حدث WebSocket إلى المتصفح. لوحة التحكم (React) تستخدم خطاف (Hook) للإنصات لهذا الحدث، وتُحدث بيانات `TanStack Query` فوراً في الذاكرة. النتيجة: التنبيه يظهر في الشاشة لحظة وقوعه على جهاز العميل بأقل من ثانية زمن انتقال.

5. لماذا يوجد محرك استجابة محلي (Local Responder) داخل الوكيل إذا كان الخادم قادراً على الاستجابة؟

الإجابة: الاستجابة المحلية تُغطي حالة "انقطاع الشبكة" أو "هجوم الفدية السريع". هجمات الـ Ransomware قد تُشفر آلاف الملفات في ثوانٍ وتفصل الشبكة. المحرك المحلي في الوكيل يفحص توقيعات الملفات (Hashes) عند تحميلها. إذا كان الملف خبيثاً وموجوداً في القائمة المحلية، يقوم الوكيل بِحَجره فوراً في أقل من 150 ميلي ثانية، دون انتظار إرسال الحدث للخادم وانتظار أمر الحجر.

6. كيف تضمنون سرية كلمات المرور ومفاتيح الـ JWT في لوحة التحكم؟

الإجابة:

- كلمات المرور لا تُخزن أبدًا كنصوص؛ بل كـ Hashes باستخدام `bcrypt` مع عامل تكلفة يمنع هجمات القوة الغاشمة.
- الـ JWT Tokens مُوقعة بمفتاح RSA خاص لا يُغادر الخادم أبدًا.
- لحماية الـ JWT من هجمات XSS، يمكن إرساله كـ `HttpOnly Cookie` .
- عند تسجيل الخروج، يتم وضع الرمز في قائمة سوداء في (Token Blacklisting) Redis حتى تنتهي صلاحيته لضمان عدم إمكانية إعادة استخدامه إذا سُرق.

7. كيف تديرون حالة النظام (State) في React مع هذا الكم الهائل من البيانات المتغيرة؟

الإجابة: بدلاً من استخدام Redux الذي يتطلب كتابة كود معقد وإدارة يدوية للـ API، استخدمنا **TanStack Query**. هو يفصل بين "بيانات الخادم" و"بيانات الواجهة المحلية". نحن نُعرِّف فقط "المفتاح" (مثلاً `['agent-events', agentId]`) ونوفر الدالة الجالبة (Fetcher). المكتبة تتكفل بالكاش، إعادة المحاولة عند الخطأ، منع الطلبات المكررة في نفس اللحظة (Deduplication)، ومزامنة البيانات في الخلفية. هذا ما يجعل الواجهة سريعة جدًا وسلسة.